

Student Query Understanding - Technical Report

1. Problem interpretation

The goal is to build an AI-powered teaching assistant that can understand a student's free-form question and classify it along three dimensions:

- **intent** - what kind of help the student seeks:
 - `Explanation`, `Example`, `Doubt clarification`, `Revision`
- **topic** - which ML concept the question is about (e.g., `Backpropagation`, `Neural Networks`, `Optimization`)
- **difficulty_level** - the student's current level or depth requested:
 - `Beginner`, `Intermediate`, `Advanced`

The system must combine **sentence embeddings**, **ML models**, and **LLM prompting** in a clean, modular way and expose a simple interface that takes a question string and returns a JSON prediction.

2. Data source and synthetic data creation

No labeled dataset exists directly for this exact task, so the solution uses a **synthetic dataset** tailored to the three label spaces above.

- **Label schema**
 - `intent` $\in$ {Explanation, Example, Doubt clarification, Revision}
 - `topic` $\in$ {
Neural Networks,
Backpropagation,
Optimization,
Gradient Descent,
Regularization,
Convolutional Neural Networks,
Recurrent Neural Networks,
Overfitting
}
 - `difficulty` $\in$ {Beginner, Intermediate, Advanced}
- **Synthetic generation procedure**
 - Implemented in `src/data\_prep.py`.
 - For each topic, the script generates multiple natural-sounding, student-style queries that vary by:
 - intent (e.g., "Can you explain...", "Can you give an example...", "I'm confused about...", "I want to revise...")
 - difficulty (e.g., "in simple terms", "more detailed explanation", "more mathematical/advanced revision").
 - This yields a compact but dense grid of examples that cover all intent and difficulty combinations for each topic.
 - The resulting dataset is shuffled and written to:
 - `data/student\_queries\_labeled.csv`
 - Columns: `text`, `intent`, `topic`, `difficulty`
- **Use of allowed public datasets**
 - The current implementation is fully functional using synthetic data only.
 - In an extended version, questions from allowed datasets such as **SciQ** or **AI2 ARC** could be mapped to approximate topics and difficulty levels (e.g., based on subject tags, question length, or curriculum level) and merged into the same schema.

- This design choice keeps the system within the constraints (≤ 50 MB data, allowed public sources) while being easy to extend with additional labeled data as needed.

- ****Assumptions****

- Student queries in practice follow patterns similar to the templates (e.g., "I don't understand X", "Can you give an example of Y?").
- A limited topic set is sufficient to demonstrate the behavior for the assignment.
- Difficulty can be inferred from phrasing (e.g., "simple terms" vs. "mathematical explanation").

- ****Limitations****

- Synthetic queries may not capture the full diversity and noise of real student language.
- Topic coverage is focused on foundational deep learning concepts; questions outside this domain will be forced into the closest available topic.
- Difficulty labels are approximate and based on wording rather than real performance data.

3. Model architecture

The system is decomposed into clear layers:

1. ****Embedding layer (`src/embeddings.py`)****

- Uses a pre-trained sentence transformer:
 - `sentence-transformers/all-MiniLM-L6-v2`
- Functions:
 - `encode\_texts(texts: Iterable[str]) -> np.ndarray`
 - `encode\_single(text: str) -> np.ndarray`
- The model is loaded once and cached with `lru\_cache` to avoid repeated initialization.

2. ****ML classification layer (`src/models.py`)****

- Reads the synthetic dataset from `data/student\_queries\_labeled.csv`.
- Computes sentence embeddings for all texts.
- Trains three independent ****Logistic Regression**** classifiers (from `scikit-learn`):
 - intent classifier
 - topic classifier
 - difficulty classifier
- Each classifier is trained and evaluated with a train/validation split; a `classification\_report` is printed.
- Trained models are persisted using `joblib`:
 - `models/intent\_clf.joblib`
 - `models/topic\_clf.joblib`
 - `models/difficulty\_clf.joblib`

3. ****Inference layer (`src/inference.py`)****

- Provides a single entry function:
 - `predict\_query(query: str) -> Dict[str, object]`
- Steps:
 - Encode the query into a sentence embedding.
 - Load the three trained classifiers (cached with `lru\_cache`).
 - Predict:
 - `intent`
 - `topic`

- `difficulty\_level`
- Optionally compute probability distributions for each classifier (`predict\_proba`) and return them under `raw\_scores` for explainability.
- This layer is **pure ML** and does not call any LLM.

4. **LLM refinement layer** (`src/llm\_refiner.py`)

- Uses the OpenAI Python client (`openai` library via the `OpenAI` client).
- Constructs a refinement prompt that includes:
 - The original student question.
 - The ML predictions and (optionally) confidence scores.
 - The allowed label sets for each dimension.
- The prompt instructs the LLM to:
 - Keep predictions that are consistent with the question.
 - Correct any field (intent, topic, difficulty_level) that is clearly inconsistent.
- Function:
 - `refine\_with\_llm(query: str, ml\_output: Dict[str, Any], model: str = "gpt-4.1-mini")`
 - Returns a dictionary containing:
 - `"ml\_prediction"` - raw ML output
 - `"final\_prediction"` - LLM-refined labels
 - For robustness, if the LLM output cannot be parsed as JSON, the ML prediction is returned unchanged.

5. **Interface / CLI layer** (`main.py`)

- Provides a simple command-line interface:
 - `--query` - required student question string
 - `--use-llm` - optional flag to activate the LLM refinement layer
 - `--llm-model` - optional model name for the LLM
- Behavior:
 - Always runs the ML-only `predict\_query`.
 - If `--use-llm` is set, passes the ML output and query into `refine\_with\_llm` and prints the combined result.
 - Otherwise, prints just the ML prediction.

5. Sample outputs

Example 1:

Input query:

```
> "I don't understand backpropagation."
```

ML-only prediction:

```
```json
{
 "intent": "Explanation",
 "topic": "Backpropagation",
 "difficulty_level": "Intermediate",
 "raw_scores": { "...": "..." }
}
```
```

After LLM refinement:

```
```json
{
 "ml_prediction": {
 "intent": "Explanation",
 "topic": "Backpropagation",
 "difficulty_level": "Intermediate",
 "raw_scores": { "...": "..." }
 },
 "final_prediction": {
 "intent": "Explanation",
 "topic": "Backpropagation",
 "difficulty_level": "Intermediate"
 }
}
```
```

Example 2:

Input query:

> "Can you give me a simple numerical example of gradient descent?"

Likely refined prediction:

```
```json
{
 "intent": "Example",
 "topic": "Gradient Descent",
 "difficulty_level": "Beginner"
}
```
```

6. Assumptions and limitations

- The system is designed to showcase:
 - Clean decomposition into ML vs. LLM components.
 - A realistic but synthetic data pipeline.
 - A clear JSON-based interface suitable for integration into an EdTech platform.
- Real-world deployment would require:
 - More diverse, real student queries.
 - Additional topics and subject areas.
 - Calibration of difficulty using behavior data (e.g., quiz scores, attempt counts).